Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №5
«Файловое хранилище»

Подготовила: Студент гр. 410902
Кухто В.С.

Проверила: Болтак С.В.

Минск 2026

Условие задания: Необходимо реализовать удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP. При реализации разрешается использовать веб-фреймворк, реализующий HTTP-роутинг и работу с HTTP-вызовами.

Путь URL запроса определяет логическое расположение файла в хранилище:

http://storage.com/path/to/file.txt - описывает файл file.txt, находящийся по пути /path/to.

Список поддерживаемых функций:

загрузка файла в хранилище с перезаписью с помощью метода PUT;

получение файла из хранилища с помощью метода GET;

получение списка файлов каталога с помощью метода GET (в качестве формата данных для ответа рекомендуется использовать JSON);

получение информации о файле (размер в байтах и дата последнего изменения) в виде HTTP-заголовков без получения содержимого файла с помощью метода HEAD;

удаление файла/каталога из хранилища с помощью метода DELETE.

Необходимо корректно использовать коды состояния HTTP.

Для тестирования созданной службы использовать утилиту curl или аналоги (Postman, HTTPie, etc). Получение файла или содержимого каталога также должно быть возможным через браузер.

Дополнительное задание

Реализовать функцию копирования файлов в новое расположение.

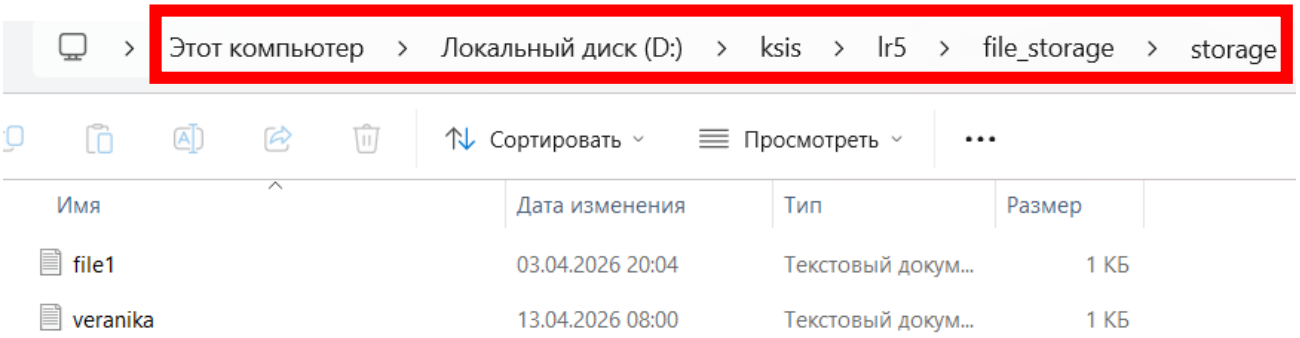
Пример API (допустимы другие варианты на усмотрение автора):

PUT http://storage.com/path/to/file.txt с установленным заголовком X-Copy-From: /path/toanother/file2.txt для копирования файла

Выполнение работы:

Написание было выполнено на языке Python. Разработка файлового хранилища происходила в среде Anaconda, в приложении Spyder. Запустить программу можно, скачав файл storage_server.py, bat-файл start_server.bat в одну папку, затем запустив файл start_server.bat (для запуска сервера). Для тестирования использовалось приложение Postman. Получение файла или содержимого каталога также возможно через браузер по ссылке http://localhost:5000.

На скриншоте ниже можно увидеть, где физически лежит папка с файлами хранилища на моем ноутбуке (далее на скриншотах я буду вести работу с предварительно очищенным от этих файлов хранилищем). Все файлы сохраняются в папку storage (туда же сохраняются каталоги и файлы в них и т.д.).



На рисунке 1 можно увидеть, как выглядит запущенный сервер (запускаю через bat-файл для удобства), тут же видно расположение файлов хранилища на моем ноутбуке (ранее было на скрине):

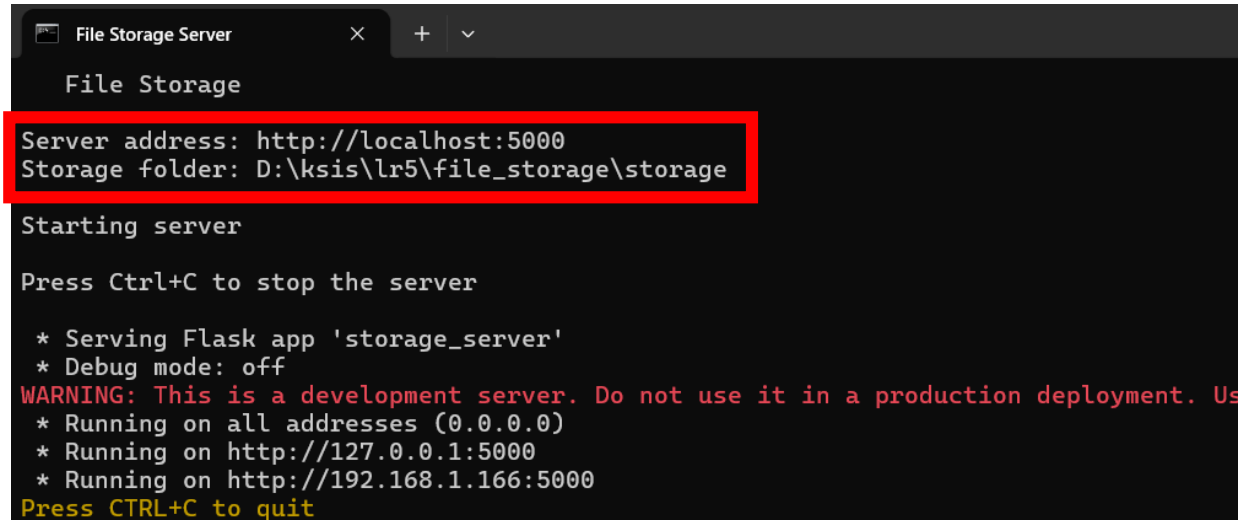


Рисунок 1

Информация красного цвета - это служебные сообщения сервера, которые показывают, что Flask запустил приложение, режим отладки выключен, сервер доступен локально в моей сети.

1 Загрузка файла

Протестирую загрузку файла в приложении Postman. На рисунке 1.1 можно увидеть содержимое файла myfile.txt, который хочу загрузить:

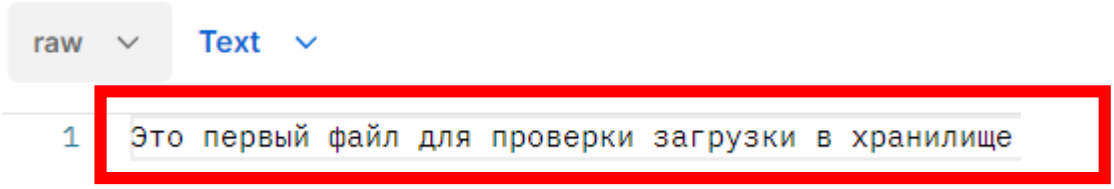


Рисунок 1.1

На рисунке 1.2 можно увидеть, какой http-запрос я отправляю на сервер: метод PUT, URL: http://localhost:5000/myfile.txt, заголовков нет, тело (само содержимое файла). В коде программы такой путь согласно заданию преобразуется в ./storage/myfile.txt на диске.

Рисунок 1.2

На рисунке 1.3 можно увидеть http-ответ. Код ответа 201 говорит о том, что ресурс успешно создан.

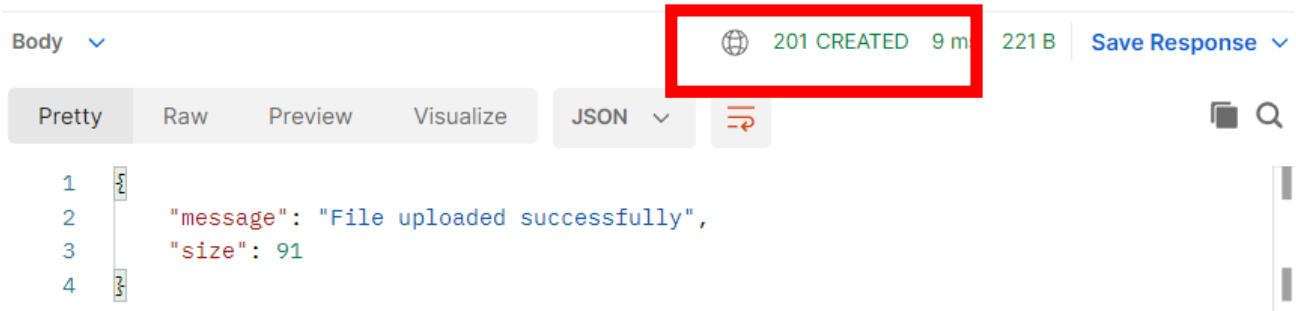


Рисунок 1.3

Также можно увидеть тело ответа в формате JSON с 2 полями "message": "File uploaded successfully" - сообщение об успешной загрузке, "size": 91 - размер загруженного файла в байтах

```
{
    "message": "File uploaded successfully",
    "size": 91
}
```

В консоли при этом можно увидеть следующий лог (рисунок 1.4). Лог сервера подтверждает, что запрос был успешно обработан (эти логи выводит сам фреймворк Flask):

```
127.0.0.1 - - [03/Apr/2026 11:44:30] "PUT /myfile.txt HTTP/1.1" 201 -
```

Рисунок 1.4

В вайшарке при этом можно увидеть следующие пакеты (1-запрос, 2-ответ), рисунок 1.5. В ответе присутствует аналогичный код 201, а также написан формат содержимого ответа (JSON):

HTTP	392	PUT /myfile.txt HTTP/1.1 (text/plain)
HTTP/J…	95	HTTP/1.1 201 CREATED , JSON (application/json)

Рисунок 1.5

Как видно на рисунке 1.6, запрос идет от клиента (в данном случае Postman, порт 55651) к серверу на порту 5000:

6	0.000899	127.0.0.1	127.0.0.1	HTTP	392 PUT /myfile.txt HTTP/1.1 (text/plain)

```
Frame 6: Packet, 392 bytes on wire (3136 bits), 392 bytes captured (3136 bits) on interface \Dev
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 55651, Dst Port: 5000, Seq: 1, Ack: 1, Len: 348
  Source Port: 55651
  Destination Port: 5000
```

```
0000  02 00 00
0010  7f 00 00
0020  90 ea 07
0030  2f 6d 79
0040  2f 31 2e
0050  70 65 3a
0060  55 73 65
```

Рисунок 1.6

Как видно на рисунке 1.7, ответ идет от сервера к клиенту

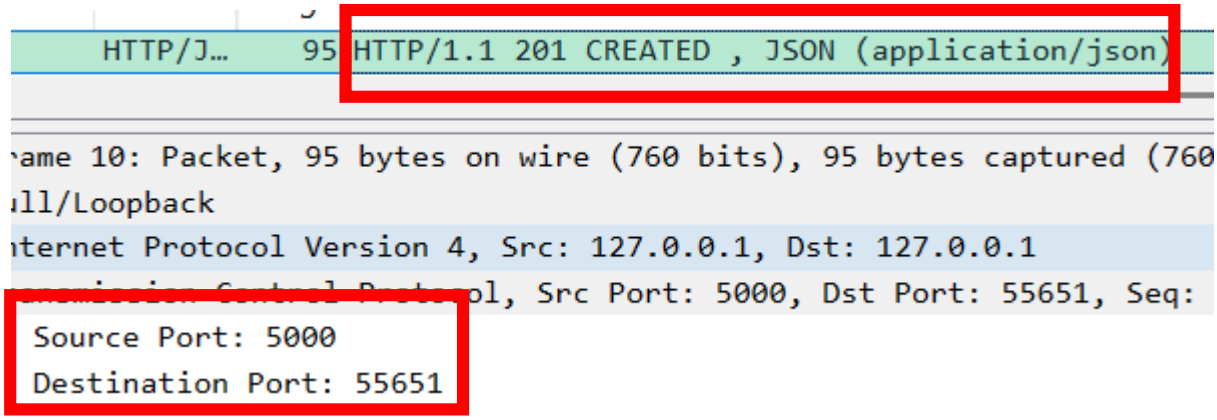


Рисунок 1.7

2 Получение файла

На рисунке 2.1 можно увидеть, какой http-запрос я отправляю на сервер: метод GET, URL: http://localhost:5000/myfile.txt.) без заголовков и тела.

Рисунок 2.1

На рисунке 2.2 можно увидеть, какой ответ я получаю:

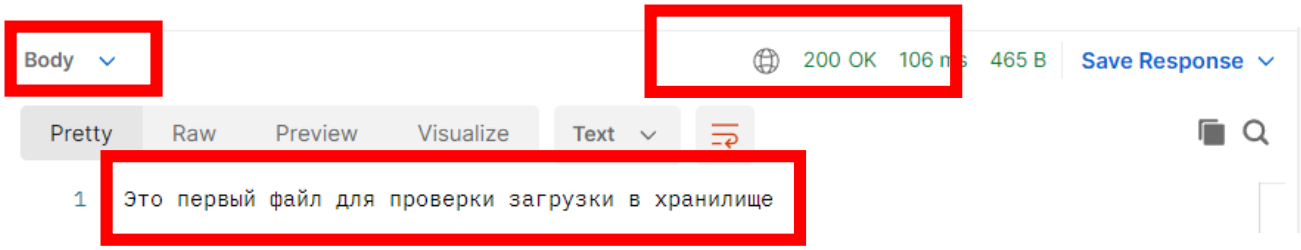


Рисунок 2.2

Как видно на рисунке выше, код ответа 200 OK – ресурс успешно предоставлен. Во кладке Body можно увидеть содержимое файла: «Это первый файл для проверки загрузки в хранилище».

В консоли можно увидеть при этом соответствующий лог (рисунок 2.3). Дата, запрос, код ответа:

```
127.0.0.1 - - [03/Apr/2026 11:58:41] "GET /myfile.txt HTTP/1.1" 200 -
```

Рисунок 2.3

Получить файл также можно через браузер по адресу http://localhost:5000/myfile.txt (рисунок 2.4). Как видно, там отображается текстовое содержимое файла:

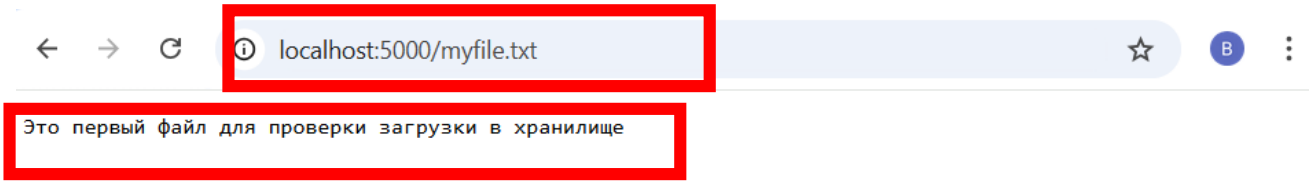


Рисунок 2.4

В вайшарке при этом можно увидеть следующие пакеты (рисунок 2.5), 1 пакет – http-запрос с GET, 2 пакет ответ с кодом 200:

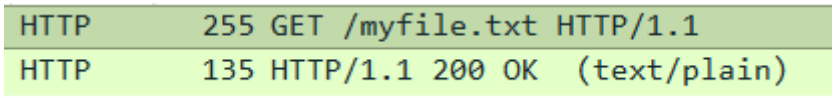


Рисунок 2.5

Раскрыв содержимое пакета с ответом, видно содержимое файла (рисунок 2.6):

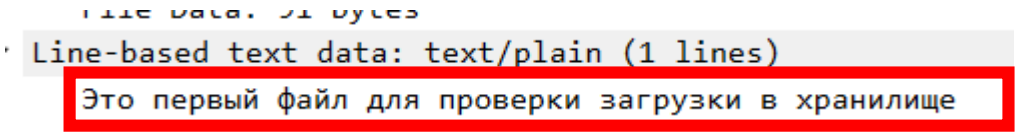


Рисунок 2.6

Также можно увидеть, какой ответ приходит при попытке получения несуществующего файла (рисунок 2.7). Я сделала GET-запрос на получение файла fileee.txt (такого нет в хранилище) и получила ответ с кодом 404 (не найдено) и сообщением в формате JSON в теле ответа о том, что файл или директория не надены:

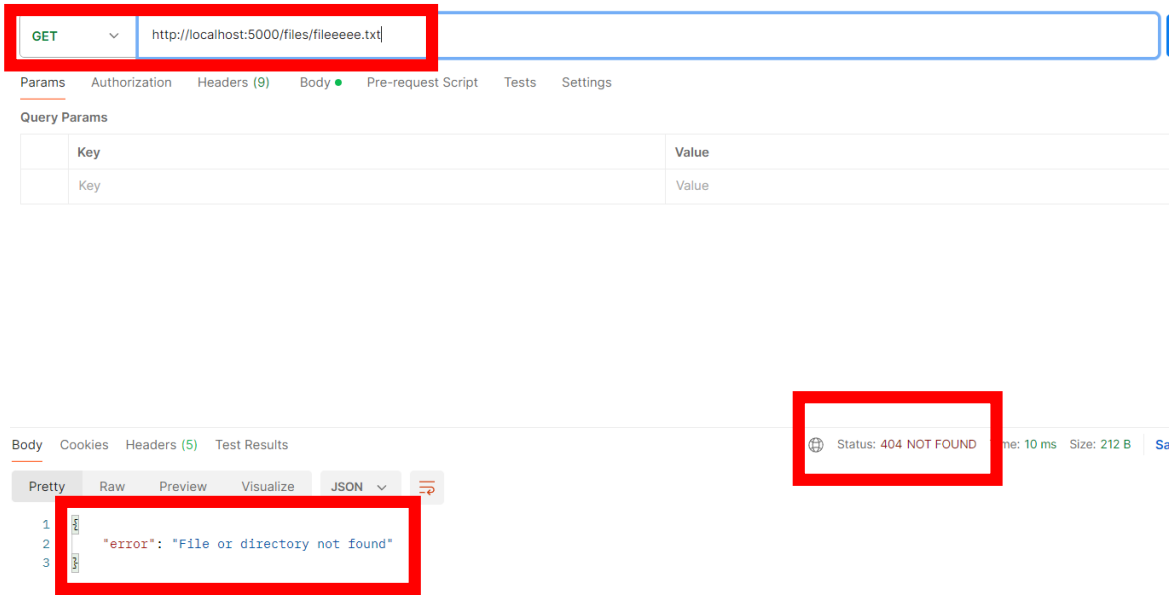


Рисунок 2.7

На рисунке 2.8 можно увидеть соответствующий лог:

```
"GET /files/fileeeee.txt HTTP/1.1" 404 -
```

Рисунок 2.8

Также можно увидеть, какой ответ приходит при неверном запросе (убрала слеш из адреса в запросе) (рисунок 2.8). Получила ответ с кодом 400 (неверный запрос) и сообщением о том, что запрошенный URL-не может быть получен:

Рисунок 2.8

3 Получение списка каталога

На рисунке 3.1 можно увидеть, какой http-запрос я отправляю на сервер: метод GET, URL: http://localhost:5000/).

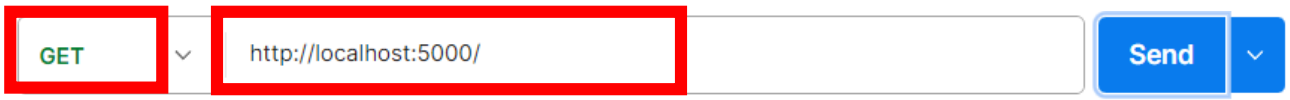


Рисунок 3.1

Содержимое ответа в формате JSON можно увидеть на рисунке 3.2. Код ответа 200 OK (ресурс успешно предоставлен). path - путь к текущему каталогу относительно корня хранилища, contents - список файлов и папок в текущем каталоге, name - имя файла или папки, type - тип объекта: "file" (файл) или "directory" (папка), size - размер файла в байтах (только для файлов), modified - дата и время последнего изменения файла:

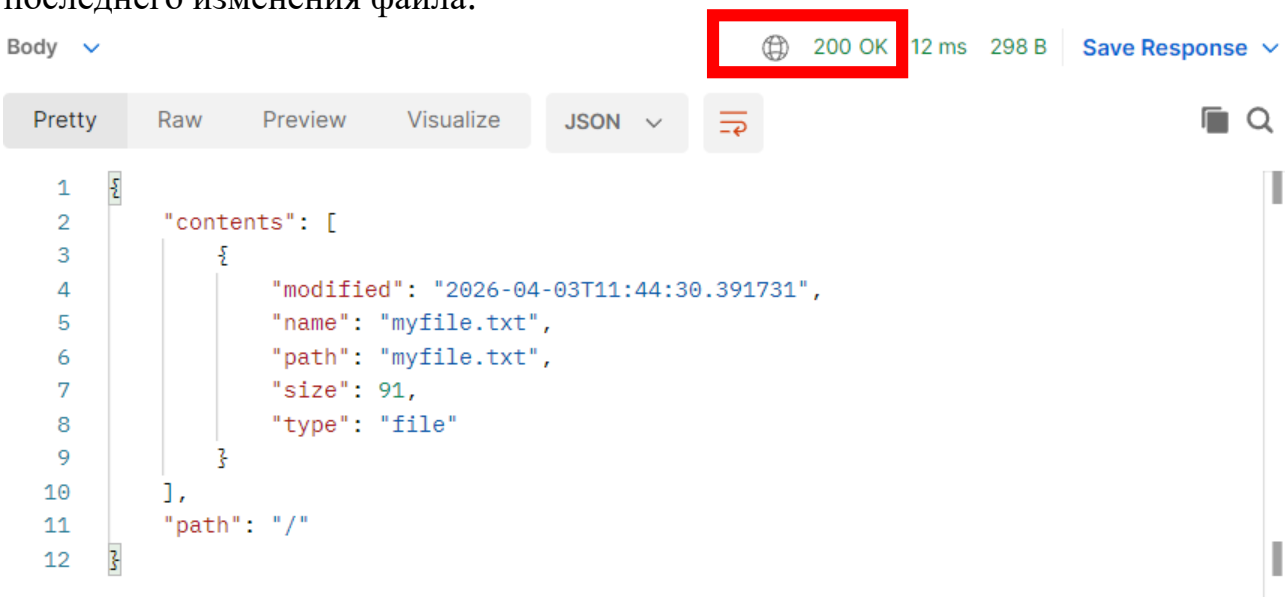


Рисунок 3.2

Соответствующий лог представлен на рисунке 3.3:

```
127.0.0.1 - - [03/Apr/2026 12:11:46] "GET / HTTP/1.1" 200 -
```

Рисунок 3.3

Также просмотреть каталог можно через браузер (рисунок 3.4). Браузер отображает тот же JSON-ответ, что и Postman:

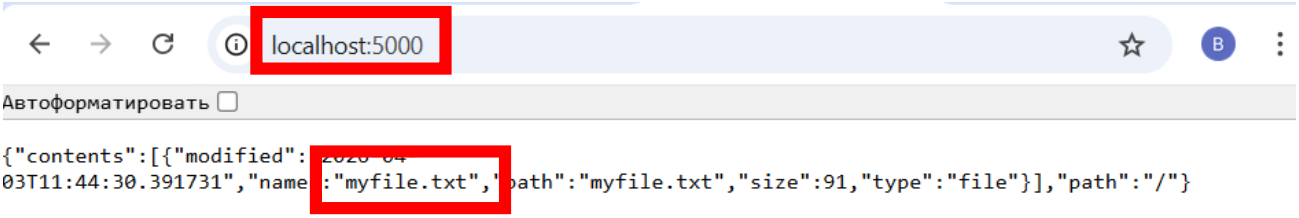


Рисунок 3.4

4 Информация о файле

На рисунке 4.1 можно увидеть, какой http-запрос я отправляю на сервер: метод HEAD, URL: http://localhost:5000/myfile.txt).

Рисунок 4.1

На рисунке 4.2 можно увидеть ответ с кодом 200 (ресурс успешно предоставлен), а также информацию в формате ключ:значение (на прошлых скриншотах разбиралась аналогичная информация об этом файле - размер файла в байтах, дата и время последнего изменения файла, тип содержимого, длина содержимого в байтах):

Headers 200 OK 3 ms 235 B

Key	Value
Server	Werkzeug/3.1.3 Python/3.13.9
Date	Fri, 03 Apr 2026 09:22:24 GMT
X-File-Size	91
X-File-Modified	2026-04-03T11:44:30.391731
Content-Type	application/octet-stream
Content-Length	91
Connection	close

Рисунок 4.2

На рисунке 4.3 соответствующий лог:

```
127.0.0.1 - - [03/Apr/2026 12:22:24] "HEAD /myfile.txt HTTP/1.1" 200 -
```

Рисунок 4.3

Пакеты в вайшарке можно увидеть на рисунке 4.4. 1 пакет – запрос с HEAD, второй пакет – ответ с кодом 200.

```
HTTP    256 HEAD /myfile.txt HTTP/1.1
HTTP    279 HTTP/1.1 200 OK
```

Рисунок 4.4

И аналогичная информация (как на скринах выше о файле) представлена в ответе на рисунке 4.5:

```
HTTP/1.1 200 OK\r\n
Server: Werkzeug/3.1.3 Python/3.13.9\r\n
Date: Fri, 03 Apr 2026 10:18:17 GMT\r\n
X-File-Size: 91\r\n
X-File-Modified: 2026-04-03T13:07:39.702581\r\n
Content-Type: application/octet-stream\r\n
Content-Length: 91\r\n
```

Рисунок 4.5

5 Удаление файла из хранилища

На рисунке 5.1 можно увидеть, какой http-запрос я отправляю на сервер: метод DELETE, URL: http://localhost:5000/myfile.txt).

Рисунок 5.1

На рисунке 5.2 можно увидеть ответ с кодом 200 (успех) и информация о том, что ресурс был успешно удален.

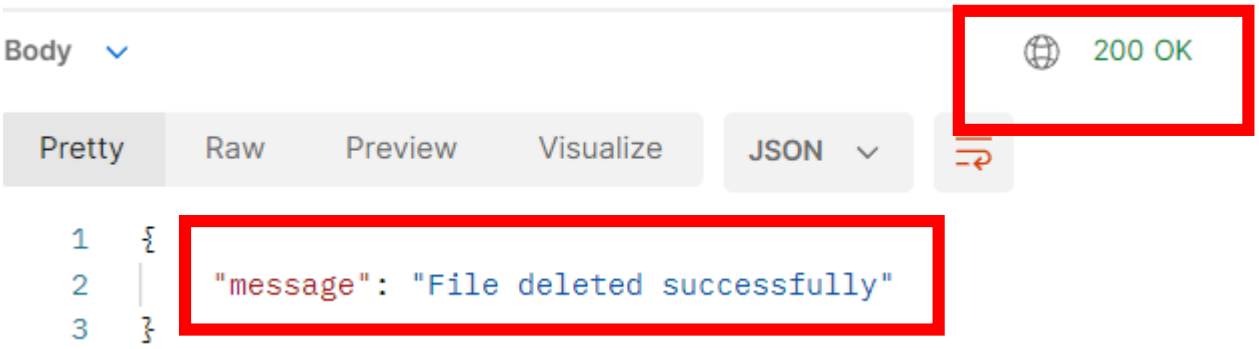


Рисунок 5.2

На рисунке 5.3 представлен соответствующий лог:

```
127.0.0.1 - - [03/Apr/2026 12:27:48] "DELETE /myfile.txt HTTP/1.1" 200 -
```

Рисунок 5.3

Теперь если мы зайдем в браузер, то этого файла там уже не будет (рисунок 5.4):

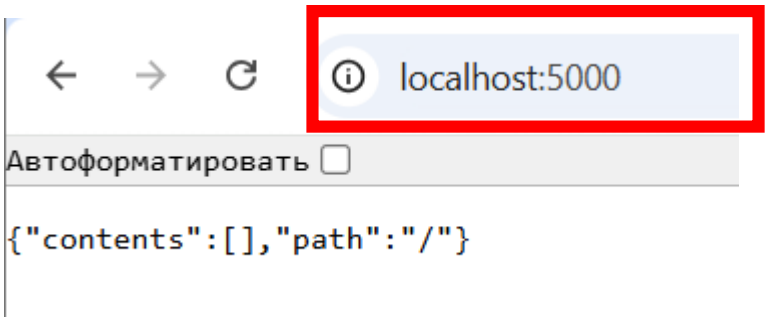


Рисунок 5.4

Пакеты в вайшарке представлены на рисунке 5.5 (запрос типа DELETE и ответ с кодом 200 и содержимом в формате JSON):

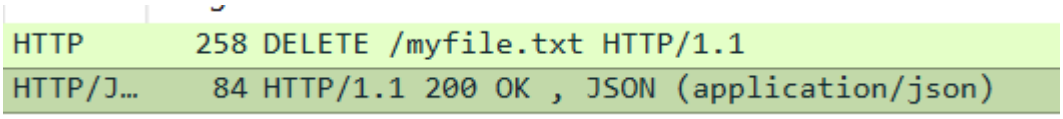


Рисунок 5.5

6 Каталог

Также продемонстрирую, как можно создать каталог в хранилище (рисунок 6.1). Создаю каталог files и файл new_file.txt в нем:

Рисунок 6.1

На рисунке 6.2 представлен ответ с кодом 201 о том, что все успешно создано:

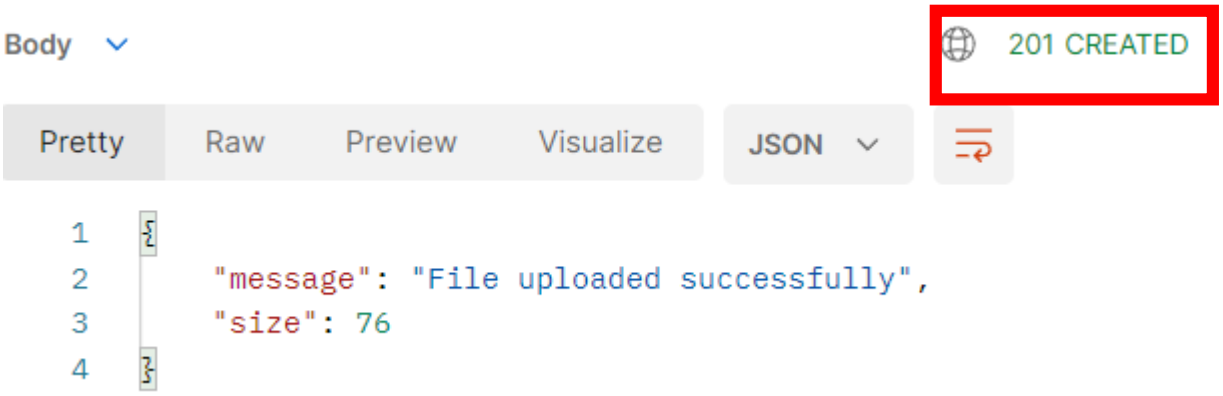


Рисунок 6.2

На рисунке 6.3 можно увидеть соответствующий лог:

```
127.0.0.1 - - [03/Apr/2026 12:34:28] "PUT /files/new_file.txt HTTP/1.1" 201 -
```

Рисунок 6.3

Просмотрим содержимое каталога в браузере (рисунок 6.4) по ссылке http://localhost:5000/files/:

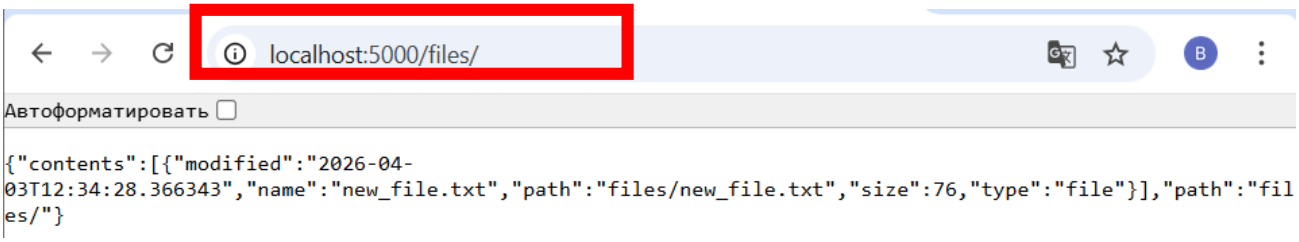


Рисунок 6.4

7 Доп.задание (копирование)

На рисунке 7.1 можно увидеть, как был создан еще один файл для демонстрации копирования. На этой картинки его содержимое до копирования:

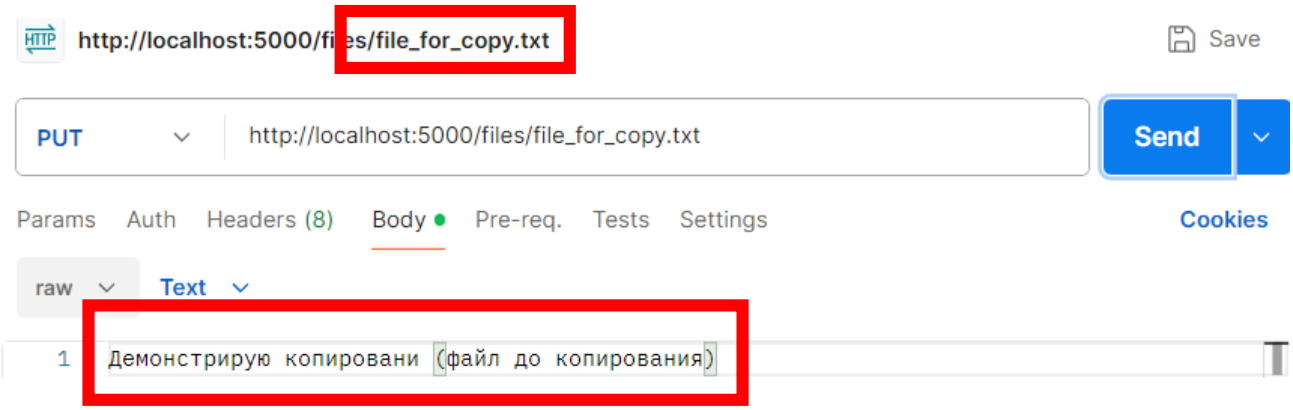


Рисунок 7.1

На рисунке 7.2 можно увидеть, какой http-запрос я отправляю на сервер: метод PUT, URL: http://localhost:5000/files/file_for_copy.txt), заголовок X-Copy-From со значением откуда информацию я хочу скопировать.

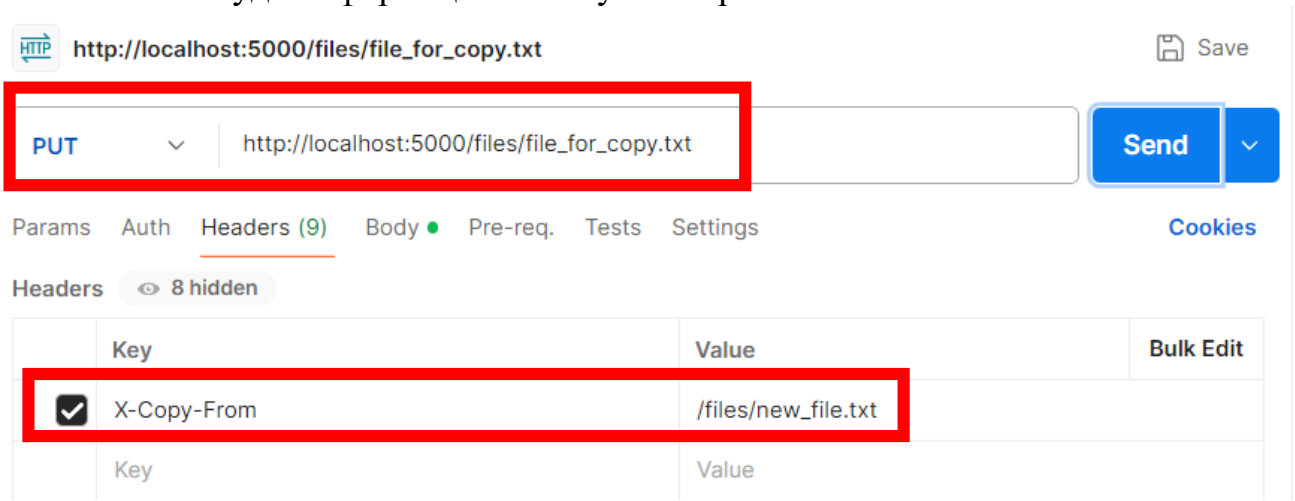


Рисунок 7.2

На рисунке 7.3 можно увидеть ответ с кодом 201 и информацией о том, что содержимое успешно скопировано:

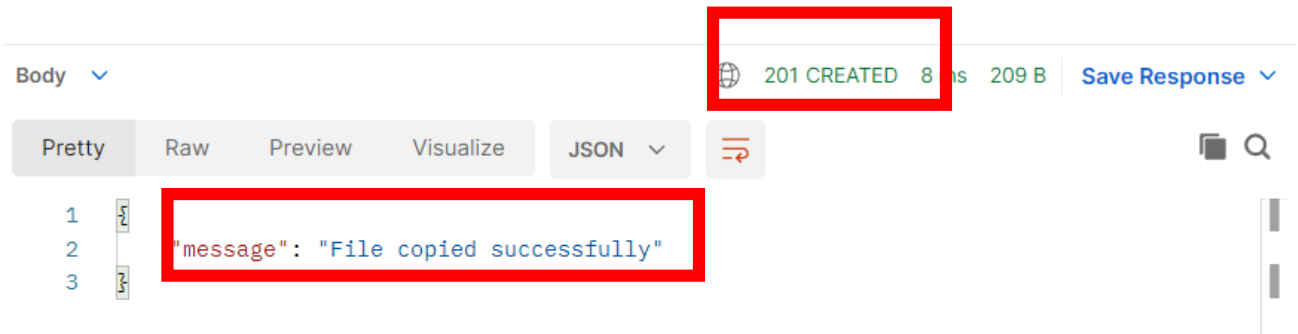


Рисунок 7.3

На рисунке 7.4 соответствующий лог:

```
2026 12:43:36] "PUT /files/file_for_copy.txt HTTP/1.1" 201 -
```

Рисунок 7.4

Теперь просмотрим содержимое файла file_for_copy (рисунок 7.5):

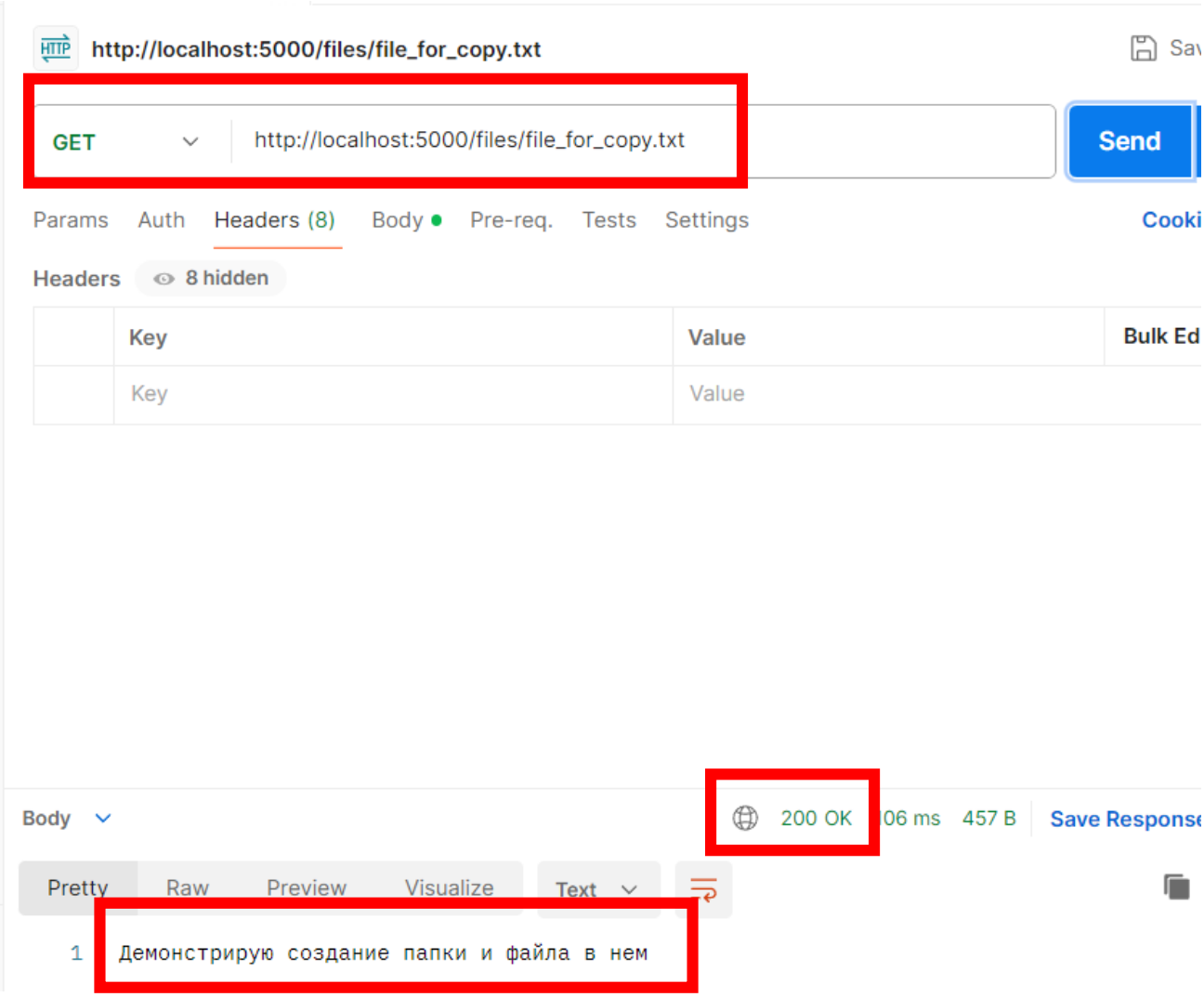


Рисунок 7.5

Удостовериться в верном копировании можно просмотрев содержимое файла, откуда копировали new_file.txt(рисунок 7.6):

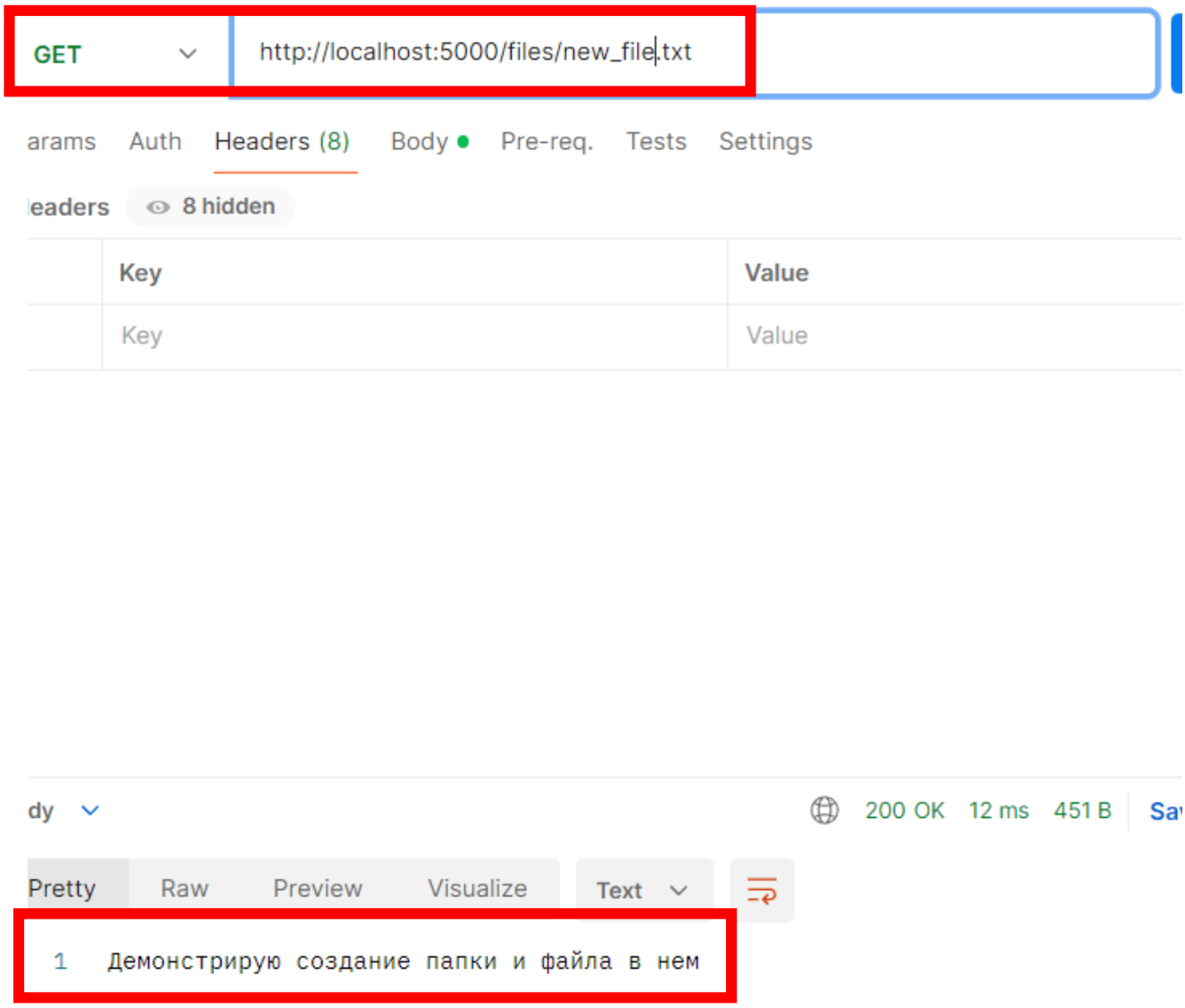


Рисунок 7.6

Пакеты в вайшарке, полученные при копировании содержимого (рисунок 7.7):

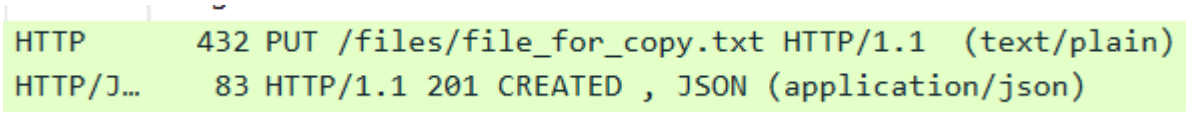


Рисунок 7.7

Содержимое ответа можно просмотреть на рисунке 7.8:

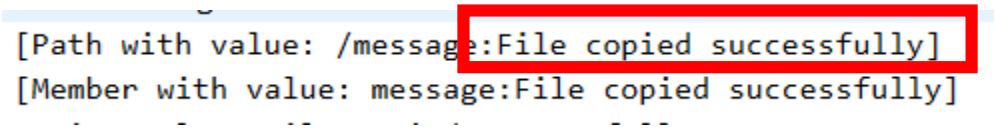


Рисунок 7.8